《人工智能与Python程序设计》—Numpy进阶知识补充

人工智能与Python程序设计 教研组

Python和Numpy进阶
知识补充

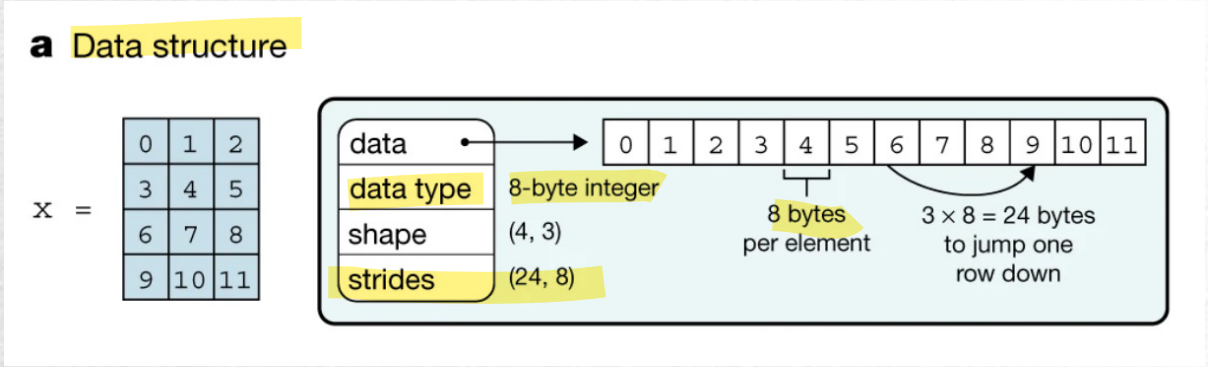
提纲

- Numpy数组的结构和索引操作

Numpy数组

- Numpy数组（ndarray）是Numpy库中最核心的数据类型
  - 支持对多维数组（又叫张量（Tensor））的高效存储和访问
  - 其结构如下：

```
In [3]: import numpy as np
        x = np.arange(12).reshape((4,3))
        print(type(x))
        print(x)

        <class 'numpy.ndarray'>
        [[ 0  1  2]
         [ 3  4  5]
         [ 6  7  8]
         [ 9 10 11]]
```



(from Array programming with NumPy, Nature volume 585, pages357–362(2020))

  - 与python内置list不同：
    - 所有元素都是一个类型的，由data type(x.dtype)决定
    - 数据保存在一段连续的内存空间中（与C语言中数组类似）
    - 目的：节省存储空间，提升访问效率

Numpy数组

- Strides=(第1个轴的元素间隔字节；第2个轴的元素间隔字节，。。。)

```
>>> x1
array([1, 2])
>>> x1.strides
(8,)
```

```
>>> x
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
>>> x.strides
(24, 8)
```

Numpy数组

- Strides=(第1个轴的元素间隔字节；第2个轴的元素间隔字节，。。。)

```
>>> y = np.random.rand(2,3,4)
>>> y
array([[[0.36660593, 0.89192705, 0.78310747, 0.28450446],
        [0.72817003, 0.93904763, 0.3960034 , 0.59260957],
        [0.36612201, 0.32366565, 0.17590845, 0.1864782 ]],

       [[0.93648374, 0.3942387 , 0.58268292, 0.30826625],
        [0.69009674, 0.04729466, 0.75724622, 0.1421677 ],
        [0.36913801, 0.94732266, 0.94664117, 0.12232551]]])
>>> y.strides
(96, 32, 8)
```

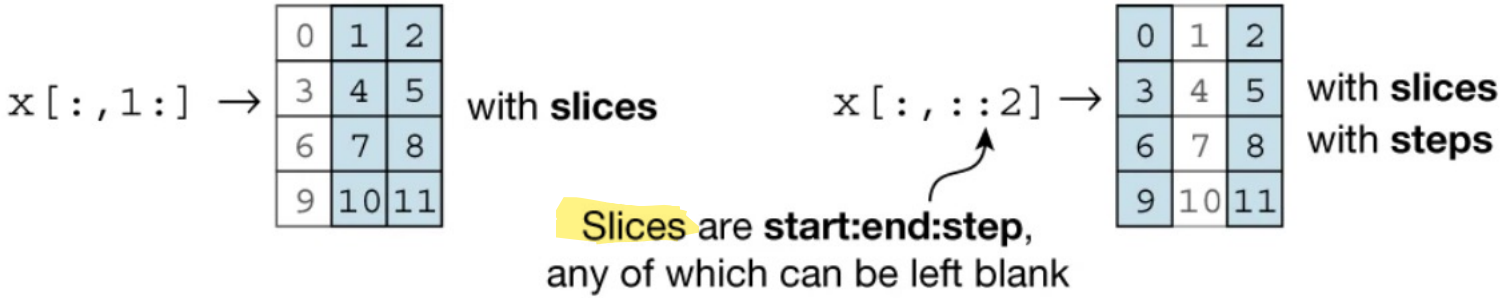
96 = 3*4*8; 32=4*8; 8

索引Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式
  - 索引一个元素：

`x[1,2]` → 5 with **scalars**

  - 使用切片（slice）索引一块子区域：



ndarray 类的索引和切片方法

- 索引的方法： x[dim1, dim2, ..., dimk, ..., dimn]
- **不建议使用 x[dim1][dim2] ...[dimk] ...[dimn]：效果不完全一样**

```
>>> x = np.random.rand(2,3,4)
>>> x
array([[[0.90159864, 0.93219804, 0.17642648, 0.32082474],
        [0.57775284, 0.2158722 , 0.14507495, 0.84780954],
        [0.69937483, 0.1823171 , 0.03817363, 0.01213278]],

       [[0.70026162, 0.10048959, 0.21653884, 0.29747806],
        [0.51585691, 0.04687248, 0.38792813, 0.84652789],
        [0.89275589, 0.75414149, 0.04726253, 0.29301265]]])
```

```
>>> x[0][1][2]
0.1450749491918859
>>> x[0,1,2]
0.1450749491918859
```

```
>>> x[:][1][:]
array([[0.70026162, 0.10048959, 0.21653884, 0.29747806],
       [0.51585691, 0.04687248, 0.38792813, 0.84652789],
       [0.89275589, 0.75414149, 0.04726253, 0.29301265]])
>>> x[:,1,:]
array([[0.57775284, 0.2158722 , 0.14507495, 0.84780954],
       [0.51585691, 0.04687248, 0.38792813, 0.84652789]])
```

ndarray 类的索引和切片方法

- 索引的方法： x[dim1, dim2, ..., dimk, ..., dimn]
- **不建议使用 x[dim1][dim2] ...[dimk] ...[dimn]：效果不完全一样**

```
>>> x = np.random.rand(2,3,4)
>>> x
array([[[0.90159864, 0.93219804, 0.17642648, 0.32082474],
        [0.57775284, 0.2158722 , 0.14507495, 0.84780954],
        [0.69937483, 0.1823171 , 0.03817363, 0.01213278]],

       [[0.70026162, 0.10048959, 0.21653884, 0.29747806],
        [0.51585691, 0.04687248, 0.38792813, 0.84652789],
        [0.89275589, 0.75414149, 0.04726253, 0.29301265]]])
```

```
>>> x[:,1,1:2]
array([[0.2158722 ],
       [0.04687248]])
>>> x[:][1][1:2]
array([[0.51585691, 0.04687248, 0.38792813, 0.84652789]])
```

回忆列表类的索引

- **只能使用 x[dim1][dim2] ...[dimk] ...[dimn]**

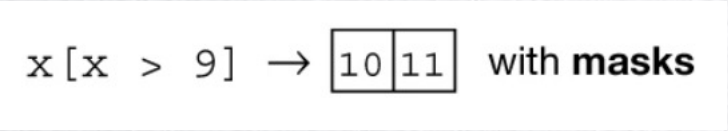
```
>>> ls
[[1, 2], [3, 4]]
```

```
>>> ls[1,3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: list indices must be integers or slices, not tup
le
```

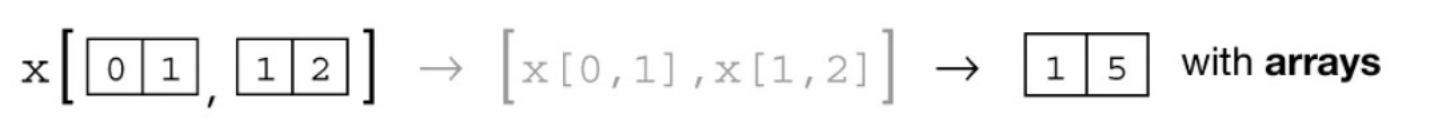
```
>>> ls[:]
[[1, 2], [3, 4]]
>>> ls[:][1:2]
[[3, 4]]
>>> ls[:][1:2][:]
[[3, 4]]
```

索引Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式
  - 按条件索引：



  - 用数组作为数组的索引：



索引Numpy数组

- Numpy数组支持比Python内置list更为丰富的索引方式
  - 用数组作为数组的索引：

```
# train / test split
shuffled_index = np.random.permutation(data_size)
x = x[shuffled_index]
y = y[shuffled_index]
```

索引|Numpy数组

- 更多例子

```
>>> import numpy as np
>>> x = np.arange(12).reshape((4,3))
>>> x
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
>>> x[::-1,::-1]
array([[11, 10,  9],
       [ 8,  7,  6],
       [ 5,  4,  3],
       [ 2,  1,  0]])
>>>
```

相当于进行了一次转置

```
>>> x
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
>>> x[2:3, -1]
array([8])
>>> x[-1,-1]
11
>>> x[-1]
array([ 9, 10, 11])
>>> x[:,-1]
array([ 2,  5,  8, 11])
>>> x[2:3]
array([[6, 7, 8]])
>>>
```

索引|Numpy数组

- 复合创建数组
  - Vstack （垂直堆叠）

```
import numpy as np
a = np.array([[1,2],[3,4]])
print 'First array:'
print a
print '\n'
b = np.array([[5,6],[7,8]])
print 'Second array:'
print b
print '\n'
print 'Vertical stacking:'
c = np.vstack((a,b))
print c
```

```
First array:
[[1 2]
 [3 4]]
Second array:
[[5 6]
 [7 8]]
Vertical stacking:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

索引|Numpy数组

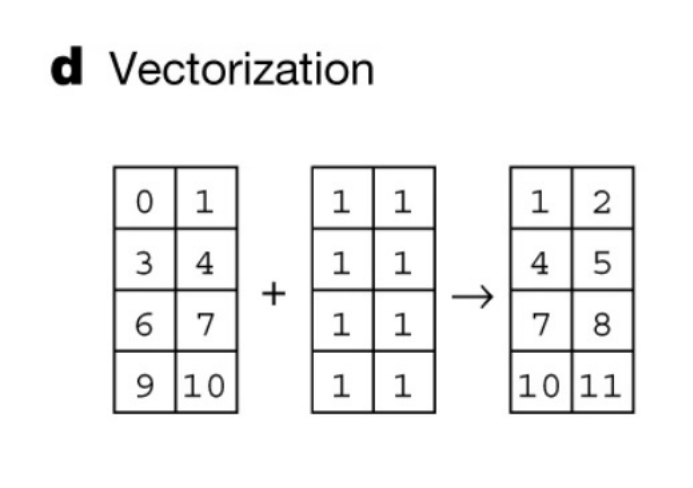
- 复合创建数组
  - hstack （水平堆叠）

```
import numpy as np
a = np.array([[1,2],[3,4]])
print 'First array:'
print a
print '\n'
b = np.array([[5,6],[7,8]])
print 'Second array:'
print b
print '\n'
print 'Horizontal stacking:'
c = np.hstack((a,b))
print c
print '\n'
```

```
First array:
[[1 2]
 [3 4]]
Second array:
[[5 6]
 [7 8]]
Horizontal stacking:
[[1 2 5 6]
 [3 4 7 8]]
```

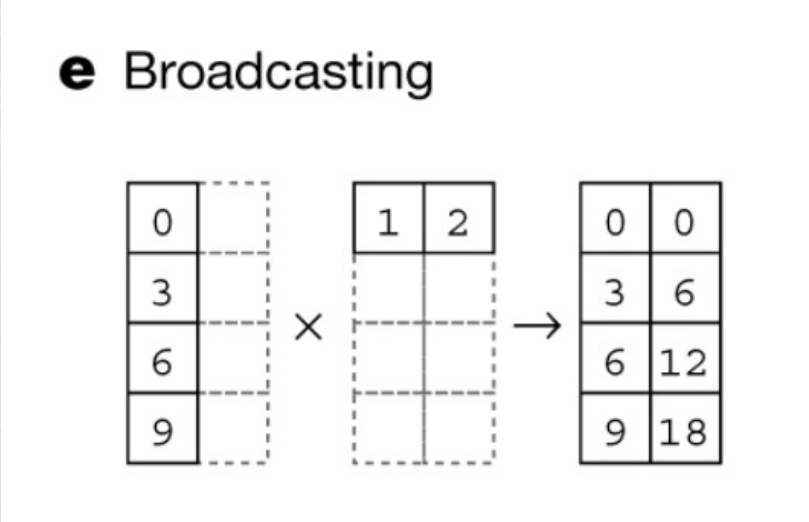
Numpy数组

- 对Numpy数组进行的操作和运算会自动的“向量化”（Vectorization）
  - 分别作用于Numpy数组中的每个元素



Numpy数组

- 广播（Broadcasting)
  - 通过“复制”d次，将1维变成d维
    - 计算效率高于使用for循环依次计算
  - 以满足向量化计算的要求



Numpy数组

- 广播（Broadcasting)
  - 通过“复制”d次，将1维变成d维
    - 计算效率高于使用for循环依次计算
  - 以满足向量化计算的要求

```
>>> x
array([[1],
       [2],
       [3],
       [4]])
>>> y
array([5, 6])
```

```
>>> x * y
array([[ 5,  6],
       [10, 12],
       [15, 18],
       [20, 24]])
```

```
>>> x + y
array([[ 6,  7],
       [ 7,  8],
       [ 8,  9],
       [ 9, 10]])
```

Numpy数组

- 广播（Broadcasting)

当运算中的 2 个数组的形状不同时，numpy 将自动触发广播机制。如：

实例

```
import numpy as np

a = np.array([[ 0, 0, 0],
              [10,10,10],
              [20,20,20],
              [30,30,30]])
b = np.array([1,2,3])
print(a + b)
```

运行结果会是什么？

Numpy数组

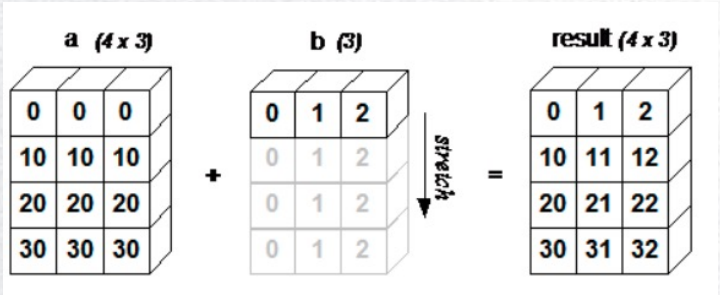
- 广播（Broadcasting)

当运算中的 2 个数组的形状不同时，numpy 将自动触发广播机制。如：

实例

```
import numpy as np

a = np.array([[ 0, 0, 0],
           [10,10,10],
           [20,20,20],
           [30,30,30]])
b = np.array([1,2,3])
print(a + b)
```

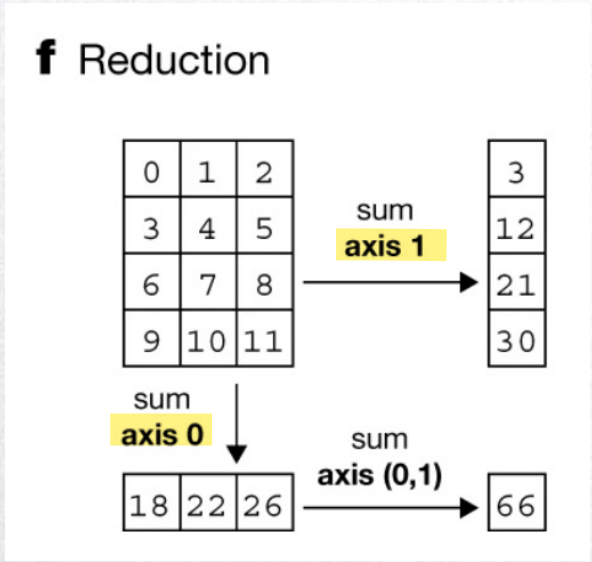
```
[[ 1  2  3]
 [11 12 13]
 [21 22 23]
 [31 32 33]]
```



Numpy数组

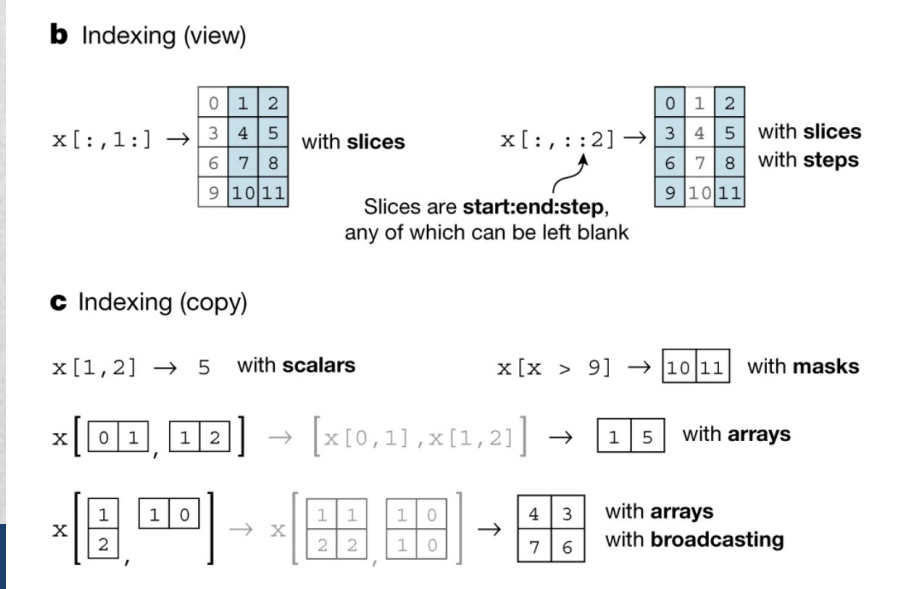
- 规约（Reduction)
  - 通过求和、求平均数等运算，将d维缩减为1维
    - 以求和函数np.sum为例
    - 可以通过axis可选参数决定按照哪个维度进行计算
    - 默认axis=None，将所有元素求和

```
In [16]: np.sum(x)
Out[16]: 66

In [17]: np.sum(x, axis=0)
Out[17]: array([18, 22, 26])

In [18]: np.sum(x, axis=1)
Out[18]: array([ 3, 12, 21, 30])
```



补充知识

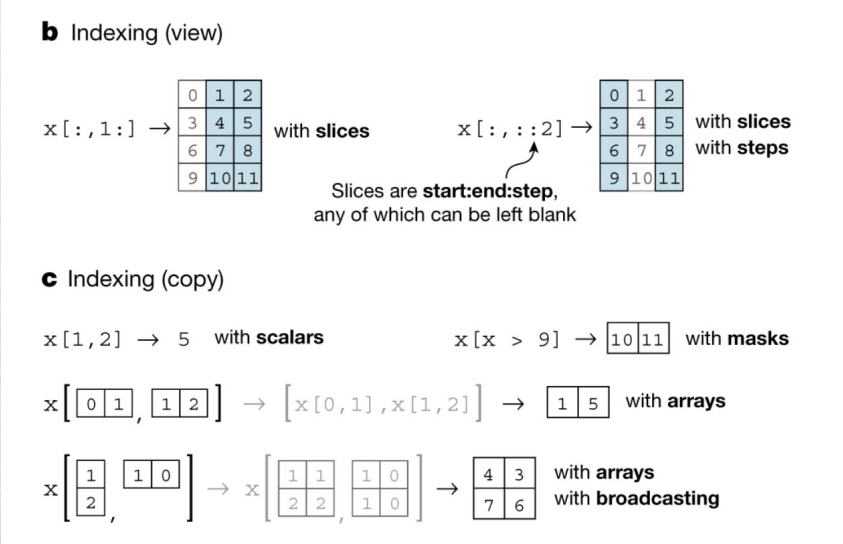
- 索引是否发生拷贝？
  - 设计思路：尽可能避免拷贝，以节省内存
  - 如果可能，索引操作会返回一个和原数组共享存储的视图（view）
  - 否则则拷贝数据生成一个新的Numpy数组对象




```
In [7]: x[:, ::2]

Out[7]: array([[ 0,  2],
               [ 3,  5],
               [ 6,  8],
               [ 9, 11]])

In [8]: x[:, ::2].strides

Out[8]: (24, 16)
```

补充知识

- 索引是否发生拷贝？
  - 设计思路：尽可能避免拷贝，以节省内存
  - 如果可能，索引操作会返回一个和原数组共享存储的视图（view）
  - 否则则拷贝数据生成一个新的Numpy数组对象



补充知识

- 索引是否发生拷贝？
  - 设计思路：尽可能避免拷贝，以节省内存
  - 如果可能，索引操作会返回一个和原数组共享存储的视图（view）
  - 否则则拷贝数据生成一个新的Numpy数组对象
    - **Strides=(第1个轴的元素间隔字节；第2个轴的元素间隔字节，。。。)**

```
>>> x
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
>>> x.strides
(24, 8)
```

```
>>> x[:,::2]
array([[ 1,  3],
       [ 4,  6],
       [ 7,  9],
       [10, 12]])
>>> x[:,::2].strides
(24, 16)
```

区别Python核心语法与第三方库

- 核心语法并不多，需要牢固掌握
  - 遇到困扰怎么办？
    - 搜索，强烈推荐stackoverflow
    - 想深入了解的同学也推荐阅读：
      - https://docs.python.org/3/tutorial/index.html
      - https://docs.python.org/3/library/index.html
      - https://docs.python.org/3/reference/index.html
- 第三方库琳琅满目：
  - turtle, numpy, torch等
  - 如何学习
    - 不要逐一背诵接口：要按照功能分类，记住最主要的一些功能（创建、索引等）
    - 要学会查表、查资料，没有一门课程能够把所有接口完全覆盖

谢谢！